

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконав(-ла):	студент(-ка) групи AI-244
	Перевознюк Ольга
	Романівна

Перевірів:	ас. Баранов О.М.
------------	------------------

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до двох моделей через ендпойнт `/api/generate`. Спочатку – просте фактичне питання, потім – задача на міркування (5 запусків для кожної моделі). Перед серією вимірів зроблено прогрівальний запит.

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ
eval_count	30	5
Час, с	199.7	5,34
Токенів/с	11.2	23.86

$$1) 199166752600/1e9 = 199,7 \text{ с}$$

$$2185/(195214745000/1e9)=11,2 \text{ токенів/с}$$

$$2) 5336139700/1e9 = 5,34 \text{ с}$$

$$5/(209588000/1e9) = 23,86 \text{ токенів/с}$$

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5/ 5	1/ 5
eval_count	1326,6	193,2
Час, с	137,54	14,04

$$1) 137\,537\,544\,160/1e9 = 137,54 \text{ с}$$

$$2) 14\,040\,505\,526/1e9 = 14,04 \text{ с}$$

Типова помилка інструктивної моделі: відповідь «10 грн», отримана без урахування того, що різниця у 100 грн розподіляється між двома доданками.

Відповіді на питання (крок 6)

1. Куди пішли додаткові токени в міркувальній моделі?

Відповідь: Величезна кількість токенів у міркувальній моделі пішла на внутрішній процес «роздумів» (Chain-of-Thought) перед видачею фінальної відповіді. Модель детально аналізує запит, будує логічні ланцюжки та перевіряє гіпотези у спеціальному прихованому або явному блоці міркувань. Для простого фактичного запитання це є надлишковим, оскільки завдання не вимагає складного синтезу.

2. На яких типах задач міркувальні моделі дають перевагу?

Відповідь: Міркувальні моделі значно перевершують інші у складних завданнях на логіку, програмування, математику та багатоступеневий аналіз даних (як показано в Таблиці 2). Їх використання є невиправданим на простих чи рутинних завданнях (наприклад, базовий переклад, коротке форматування або прості фактичні питання), оскільки це призводить до зайвої витрати часу та обчислювальних ресурсів.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: Окрім стандартного навчання на інструкціях (SFT), міркувальні моделі додатково проходять етапи навчання з підкріпленням (Reinforcement Learning), орієнтовані на оптимізацію процесу побудови логіки та внутрішніх кроків міркування. Модель заохочується за правильний шлях вирішення проблеми, а не лише за фінальний результат. Це дозволяє їй самостійно знаходити й виправляти помилки ще до виведення тексту користувачу.

4. Чи означає більша кількість токенів, що модель «розумніша»?

Відповідь: Велика кількість токенів сама по собі не робить модель «розумнішою», вона лише свідчить про наявність додаткового етапу обробки та внутрішнього аналізу. Вона буває ефективною лише для складних завдань, що підтверджується порівнянням результатів у Таблиці 2. На простих завданнях (Таблиця 1) це призводить лише до неефективної трати часу без покращення якості відповіді.

5. Як пояснити різницю у швидкості генерації між моделями?

Відповідь: Різниця у швидкості зумовлена архітектурними особливостями, кількістю параметрів та наявністю складних алгоритмів опрацювання міркувань у моделі **qwen3:4b**. Модель **llama3.2:3b** генерує текст швидше, оскільки вона

оптимізована як легка інструктивна модель без ресурсомісткого циклу попереднього побудовного аналізу.

Висновок за завданням 1

Експеримент показав, що два показники — швидкість генерації та загальний час відповіді — не можна плутати. Перший характеризує обладнання й розмір моделі, другий — обсяг роботи, яку модель собі призначила. На фактичному питанні режим міркування виявився чистою перевитратою: результат однаковий, витрати відрізняються у 6 разів. На задачі про ручку й зошит ситуація протилежна — саме розписаний хід дозволив уникнути інтуїтивно привабливої, але неправильної відповіді. Практичний висновок: тип моделі треба обирати під тип задачі, а для агента з 8 послідовними викликами різниця складе 1100,3 с проти 112,3 с, що робить міркувальну модель непридатною для налагодження.

Загальні висновки

- 1) Локальна й хмарна моделі використовують єдиний інтерфейс, тому перехід між ними не потребує змін у логіці програми – це основа стратегії резервування провайдерів у реальних проєктах;
- 2) Затримка відповіді складається з двох незалежних показників – TTFT та швидкості генерації, висновок про те, «яка модель швидша», без їх розділення є некоректним;
- 3) Режим міркування є інструментом із чіткою областю застосування: він окупається на багатокроковому виведенні й є чистою перевитратою на фактичних питаннях і переказі;
- 4) Промпт та system-повідомлення є ненадійним засобом керування форматом відповіді: пряму інструкцію проігнорували моделі різного розміру, включно з хмарною. Надійність досягається програмною валідацією, а не формулюванням;

- 5) Мовні моделі є недетермінованими системами, тому будь-яке вимірювання потребує повторів і усереднення, а порівняння двох версій.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Таблиця 3: Вплив system-повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	585	Українська	7	+
llama3.2:3b	українською, 2 речення	233	Українська	2	+
qwen3:4b	лаконічний асистент	133	Українська	2	+
qwen3:4b	українською, 2 речення	138	Українська	1	-

Таблиця 4: Витрати токенив

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	49	183	232
llama3.2:3b	українською, 2 речення	53	69	122
qwen3:4b	лаконічний асистент	52	3090	3142
qwen3:4b	українською, 2 речення	61	697	758

- 1) Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Найточнішою була Llama. Qwen написала туж саму відповідь, що і з лаконічною системою, проте крапку змінила на кому.

- 2) Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Ні, обидві моделі з різними вимогами завжди відповідали українською.

- 3) Як змінився `prompt_tokens` при зміні `system`повідомлення і чому?

Так, змінились, з обмеженнями зріс. Змінився, бо довша системна інструкція вимагає більше токенів для її зчитування моделлю.

- 4) Чому в `qwen3:4b` значення `completion_tokens` значно більше, ніж у `Llama3.2:3b`, навіть при однаковій довжині видимої відповіді?

Значення `completion_tokens` у моделі `qwen3:4b` настільки велике через те, що вона є міркувальною моделлю (Reasoning model).

- 1) Я порівнював інструктивну модель (`Llama3.2:3b`) та міркувальну (`qwen3:4b`) з різними `system`повідомленнями. Чому моделі порізноmu дотримуються прямих інструкцій щодо довжини та мови відповіді?

Різниця в дотриманні інструкцій зумовлена архітектурними особливостями, обсягом навчальних даних та способом вирівнювання (alignment) кожної моделі. Інструктивні моделі краще оптимізовані під пряме дотримання обмежень формату, тоді як міркувальні моделі можуть фокусуватися на генерації внутрішньої логіки на шкоду точного виконання зовнішніх стилістичних обмежень. Це яскраво видно в Таблиці 3, де модель `qwen3:4b` з другою інструкцією не виконала умову щодо кількості речень.

- 2) Чим `system`повідомлення відрізняється від `user`повідомлення з точки зору моделі — чи справді воно має більший пріоритет?

`System`-повідомлення задає глобальний контекст, роль, поведінкові рамки та загальні обмеження для всього діалогу. Воно дійсно має вищий пріоритет порівняно з `user`-повідомленням, оскільки в архітектурі

підготовки промπτу воно обробляється як базовий вектор інструкцій, що керує подальшою генерацією. Проте в малих моделях цей пріоритет може нівелюватися, якщо користувацьке завдання суперечить системному або вимагає складного логічного перемикавання.

- 3) Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Міркувальна модель генерує велику кількість прихованих токенів у блоці внутрішніх роздумів (Chain-of-Thought) перед виведенням результату. Як наслідок, загальна кількість `completion_tokens` різко зростає (що підтверджує Таблиця 4), хоча сам текст відповіді залишається коротким. Інструктивна модель виконує запит напряму, не створюючи додаткового проміжного шару міркувань.

- 4) Що станеться, якщо передати моделі суперечливі інструкції в `system` і `user`, — яку вона виконає?

Зазвичай модель намагається знайти баланс, але частіше надає пріоритет інструкціям із `system`-повідомлення через його глобальний статус. Водночас у малих або недостатньо налаштованих моделях свіжіші або більш детальні вказівки з `user`-повідомлення можуть перебити системні рамки. Усе залежить від ваги токенів інструкції та того, наскільки суперечливими є ці правила.

- 5) Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Малі моделі мають обмежену ємність параметрів, через що гірше утримують у пам'яті складні багатокомпонентні умови (наприклад, одночасне обмеження на мову, кількість речень і стиль). Вони більш схильні «збиватися» на ту мову, якою написане саме `user`-повідомлення або яка переважала в їхніх тренувальних датасетах для конкретного домену. Через це інструкції на кшталт «відповідай українською, 2 речення» іноді виконуються з помилками, що зафіксовано в Таблиці 3.

Висновок за завданням 1:

Експеримент показав, що вибір системних інструкцій та типу моделі критично впливає на ресурси і точність генерації. На простих запитах зміна мови та довжини промпту збільшує обсяг `prompt_tokens`, проте для міркувальних моделей головною статтею витрат є `completion_tokens` через внутрішній ланцюжок роздумів. Якщо на фактичних питаннях такий режим є надлишковою перевитратою токенів і часу, то на складних умовах саме детальна системна рамка допомагає спрямувати модель. Практичний висновок: системні інструкції не гарантують 100% виконання жорстких обмежень на малих моделях, а тому для стабільного форматування вимагають дублювання в користувацькому запиті або використання потужніших архітектур.

Загальні висновки:

- 1) System-повідомлення задає глобальні рамки та пріоритетний контекст для діалогу, проте воно не гарантує 100% виконання на рівні жорсткого коду. Малі моделі можуть ігнорувати або частково втрачати системні інструкції через обмежену ємність пам'яті та складність логічних умов.
- 2) Самої лише текстової інструкції часто недостатньо для стабільного дотримання суворих форматів (наприклад, кількості речень чи точної структури), що підтверджується експериментальними даними. Моделі можуть збиватися на звичні паттерни генерації або підлаштовуватися під мову користувацького запиту.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	gemini-3-flash
Час відповіді, с	5.06 с	7.86 с
completion_tokens	41	460
Довжина відповіді, символів	138	1198
Мова відповіді	Українська	Українська
Фактичних помилок у відповіді	2	0

- 1) Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

В коді вручну не було потреби змінювати рядки, ця логіка вже була прописана.

- 2) Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Від хмарної моделі відповідь була більш повною. Вона була довшою та розібрана по пунктам.

- 3) Чи були в локальній моделі фактичні помилки або проблеми з українською мовою?

Так, були. Були проблеми з закінченням у слів, а також не дуже вдале порівняння (на мій погляд).

- 4) Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

В моєму випадку хмарна модель була довшою. Причиною може бути: мережева затримка, черги на серверах та від самої моделі.

- 1) Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Формат запитів OpenAI став ринковим стандартом через ранній лідерство компанії та максимально продуманий, зручний інтерфейс API ([/v1/chat/completions](#)). Іншим розробникам і сервісам вигідно адаптувати свої продукти під цей формат, оскільки це дозволяє розробникам змінювати провайдерів однією строчкою коду без переписування архітектури додатків.

- 2) Я порівнював локальну модель на 3 млрд параметрів і хмарну на 70 млрд. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Кількість параметрів критично впливає на здатність моделі опрацьовувати складну логіку та уникати фактичних помилок, що чудово видно у вашій таблиці (де хмарна модель дала 0 помилок проти 2 у локальній). Проте на якість також суттєво впливають якість та обсяг навчальних даних, етап вирівнювання (alignment) та наявність системних інструкцій.

- 3) Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Примітка щодо вашої таблиці: у вашому випадку хмарна модель генерувала у рази більшу відповідь (1198 символів і 460 токенів проти 138 символів у локальній), тому її загальний час трохи більший (7.86 с проти 5.06 с). Проте загалом хмарні моделі часто відповідають швидше за рахунок потужного серверного обладнання (кластерів з GPU/TPU) та оптимізованих алгоритмів інференсу. Локальна модель на домашньому ноутбукі обмежена ресурсами вашого процесора чи відеокарти.

- 4) Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Головний ризик полягає в тому, що конфіденційні дані користувачів або код компанії передаються на сторонні сервери провайдера, де вони потенційно можуть зберігатися чи використовуватися для навчання. У комерційних проєктах для захисту приватності підписують спеціальні угоди (DPA), використовують корпоративні тарифи з гарантією незбереження даних або розгортають моделі локально.

- 5) Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Rate limit — це ліміт на кількість запитів або токенів за певний проміжок часу (наприклад, запитів за хвилину), який встановлюють провайдери для захисту систем від перевантаження. Щоб обійти це обмеження, розробники використовують черги завдань, механізми повторних спроб із затримкою (exponential backoff), балансування навантаження між кількома акаунтами або різними провайдерами.

Висновок до завдання 3:

Локальну модель варто обирати для обробки конфіденційних або чутливих даних, таких як персональні записи чи комерційна таємниця, які не можна передавати стороннім серверам, а також у проєктах, де критично важлива незалежність від інтернету та робота в умовах нестабільної мережі чи для простих рутинних завдань без сторонніх лімітів. Натомість хмарну модель краще використовувати для складних завдань, що вимагають глибокої логіки та високої точності, коли важлива швидкість генерації завдяки потужним серверним ресурсам провайдера або у випадках, коли відсутня можливість розгорнути важке обладнання на власній стороні, а зручність API та стабільна якість переважають ризики передачі даних.

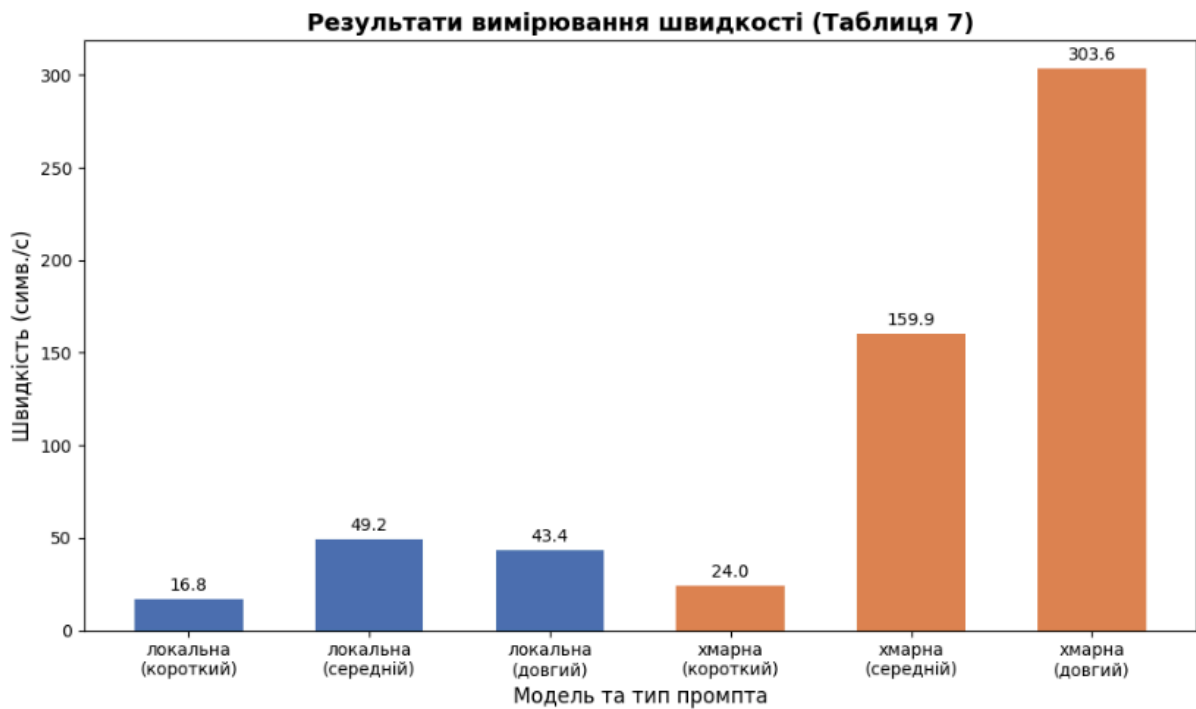
Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Таблиця 7: Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	2.128	3.409	16.8
локальна	llama3.2:3b	середній	2.113	9.333	49.2
локальна	llama3.2:3b	довгий	2.168	29.474	43.4
хмарна	gemini	короткий	1.424	1.457	24
хмарна	gemini	середній	2.635	3.4	159.9
хмарна	gemini	довгий	4.411	9.401	303.6

Таблиця 8: Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	2.128	2.126
Всього, с	3.409	2.926



Крок 6:

- 1) У якій моделі кращий TTFT і чому саме в неї?

Хмарна модель має кращий (менший) TTFT на короткому промпті (1.424 с), проте локальна модель демонструє набагато більшу стабільність показника TTFT незалежно від довжини промпту (~2.1–2.2 с). Локальна модель стабільніша, бо працює на вашому пристрої без мережових затримок, тоді як хмарна на довгих промптах витрачає додатковий час на передачу великого обсягу даних через інтернет.

- 2) Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.

Ні, не збігається. Найменший час першого відгуку (TTFT) зафіксовано у хмарної моделі на короткому промпті (1.424 с), а найвищу швидкість генерації (символів/с) показала та ж хмарна модель на довгому промпті (303.6 символів/с). TTFT залежить від швидкості появи найпершого токена та затримки з'єднання, тоді як швидкість у символах/с вимірює ефективність потокового виведення вже після старту генерації.

- 3) Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

На коротких промптах швидкості обох моделей були порівнянними (16.8 і 24 символи/с), але з переходом до довгих промптів розрив катастрофічно збільшився на користь хмарної моделі. На довгому промпті хмарна модель досягла 303.6 символів/с проти 43.4 у локальній, оскільки потужні сервери дата-центру набагато ефективніше обробляють великі обсяги контексту.

- 4) Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Три повтори можуть помітно відрізнитися через випадкові чинники: мікроколивання мережі, завантаженість процесора локального ПК або черги на серверах провайдера. Одного запуску недостатньо, оскільки він може випадково потрапити на пікове навантаження (або стати «викидом»), тому використання кількох спроб і розрахунок медіани гарантують об'єктивність результатів.

- 5) Чому холодний старт повільніший за теплий?

Холодний старт вимагає часу на первинне завантаження вагів моделі з накопичувача в оперативну або відеопам'ять, ініціалізацію середовища та виділення ресурсів. При теплому старті модель уже повністю завантажена і перебуває в активній пам'яті (кешована), що дозволяє уникнути цих накладних витрат і виконати запит значно швидше.

Крок 7:

- 1) Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

Під час TTFT (Time to First Token) модель виконує попередню обробку (*prefill*) всього вхідного промпта: токенизує текст, завантажує матриці вагів і паралельно прораховує всі токени вхідного запиту. Його вимірюють окремо,

оскільки фаза обробки входу суттєво відрізняється від процесу покрокової генерації тексту, а для користувача цей показник визначає суб'єктивну «реактивність» інтерфейсу.

- 2) Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Гірший TTFT у хмарної моделі зазвичай спричинений мережевою затримкою (*network latency*) на передачу запиту через інтернет, чергами на серверах провайдера та часом на первинну обробку великого обсягу даних. Проте після старту потужні серверні кластери з GPU/TPU генерують подальші токени набагато швидше завдяки високій обчислювальній потужності, яку не може забезпечити звичайний домашній ноутбук.

- 3) Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Перший запуск — це **холодний старт**, під час якого система змушена фізично завантажувати гігабайти вагів моделі з диска в оперативну чи відеопам'ять та ініціалізувати середовище виконання. Наступні заклики є «теплыми», оскільки модель уже повністю завантажена в пам'ять (кешована), що дозволяє уникнути накладних витрат на читання з накопичувача.

- 4) Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Окремі виміри можуть сильно коливатися через випадкові чинники: мікрозатримки мережі, фонові процеси операційної системи, завантаженість процесора або тимчасові черги на серверах хмарного провайдера. Використання кількох запусків і розрахунок медіани дозволяють нівелювати випадкові «викиди» та отримати об'єктивний, достовірний результат.

- 5) Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Обробка вхідного запиту (*prefill*) виконується **паралельно** — апаратне забезпечення (GPU) може проаналізувати всі токени готового промпта одночасно за один або кілька масивних кроків. Натомість генерація відповіді (*decoding*) є **авторегресійною**: модель змушена створювати текст суворо послідовно, по

одному токеноу за раз, щоразу додаючи новий елемент до контексту і прораховуючи мережу наново.

Завдання 5. Оцінювання якості моделей за рубрикою

Таблиця 13: Оцінка якості за рубрикою

Завдання	Лlama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	0	1	2
2. Формат	0	0	0
3. Стиснення	1	0	1
4. Код	2	0	2
5. Логіка	0	0	0
6. Українська	1	?	2
Разом /12	3	1	7

З моделлю qwen виникла проблема з відмовами відповідати та їй було замало часу, хоча діапазон часу я збільшила до 300 секунд. На питання з української мови, не маю змогу відповісти, бо вона дала відповідь тільки на 1 питання з роком і лише 1 раз.

Таблиця 14: Розбіжність оцінок між двома прогонами (так / ні)

Завдання	Лlama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	+	+	-
2. Формат	+	-	+
3. Стиснення	+	-	+
4. Код	-	-	+
5. Логіка	+	-	+
6. Українська	+	-	+

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Найбільшу кількість балів набрала хмарна модель. Насправді, різниця виникла майже в усіх аспектах. тільки за пунктом 4, хмарна модель та модель Лlama отримали хороші результати.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Розійшлися теж майже в усіх. Це свідчить про не надто надійну модель, що змінює свою відповідь з кожним разом.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Ні. Усі хибили.

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

В хмарної моделі проблем не виникало. Проте Llama мала, як русизми, так і взагалі слова з інших мов.

Тест 6 завдання моделі Llama:

```
"6_language": [
    "Документальне кіно відрізняється від художнього кіно такими відмінностями:\n\n1. Цельності: Документальне кіно зосереджено на передачі реальності та подій, які відбуваються у світі, тоді як художнє кіно є творською формою, що додає до життя реальності свої емоції та естетичні прикраси.\n2. Стиснися truthfulness: Документальне кіно є такою формою, що цілесно постачає ефектність truthfulness та точності, тоді як художнє кіно є відмінним формою, що стислює емоційне зображення повсякденного життя.",
    "Документальне кіно відрізняється від художнього своїм метою та змаганням:\n\n1. Мета документального кіно полягає в зароблінні реальності, передачі історії та подій, тоді як художнє кіно фокусується на творчому висловлінні, експресії та емоційному впливі."
]
```

Тут ми можемо побачити проблеми з використанням української мови. Причиною цього може бути дисбаланс даних. Більша частина сайтів написана з англійською мовою, тому в моделі може виникати проблема з перекладом на іншу мову. Так само і питання з русизмами. Через те, що мови мають спільні літери, а інколи і слова, то модель плутає мови.

1. Я оцінював мовні моделі за рубрикою з шести завдань і трибальною шкалою. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Окрім експертної чи ручної оцінки за рубриками, в індустрії застосовують такі підходи, як автоматичні бенчмарки для тестування моделей на стандартизованих наборах даних для конкретних завдань, підхід «модель у ролі судді» з використанням

потужніших моделей для автоматичного оцінювання, краудсорсинг та сліпе A/B тестування на спеціальних платформах для формування глобального Elo-рейтингу, а також метрики подібності для автоматичного порівняння еталонних і згенерованих відповідей.

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

Це відбувається через стійкі звички з навчальної вибірки, де мільярди разів код і структуровані дані подавалися всередині блоків markdown як патерн красивого оформлення, а також через загальну слабкість негативних обмежень у нейромережах, яким набагато важче виконувати заборони порівняно з прямими вказівками.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Головними причинами є дисбаланс навчальних даних, оскільки основний масив якісних текстів є англомовним, неефективна токенизація, що гірше розбиває українські слова на окремі одиниці, а також складна морфологія української мови з розгалуженою системою відмінків і форм.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Це пов'язано зі стохастичністю та імовірнісною природою генерації тексту, де модель на кожному кроці видає розподіл імовірностей для всього словника, а за встановленої температури вибір наступного слова відбувається випадково, створюючи різні шляхи побудови речення.

5. Що таке галюцинація мовної моделі і чому модель радше вигідає факт, ніж визнає незнання?

Галюцинація — це ситуація, коли модель упевнено генерує неправдиву чи вигадану інформацію, видаючи її за правду через те, що її архітектура спочатку проєктувалася як статистичний предиктор наступного токена для створення плавного і правдоподібного тексту, а не для звірки фактів з базою даних чи чесною відмови від відповіді без спеціального додаткового навчання.

Завдання 6. Керування випадковістю: temperature і top_p

Таблиця 16: Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	1	0
temperature	0.7	5	—
temperature	1.5	5	1
top_p	0.1	1	—
top_p	0.5	5	—
top_p	1.0	5	—

Таблиця 17: Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так

Крок 7:

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Для агента, який має генерувати суворий JSON, потрібно ставити temperature = 0.0. Це необхідно тому, що завдання вимагає максимальної точності, дотримання синтаксису, структури та ключів.

2. Яке — генератору варіантів заголовків?

Для генератора варіантів заголовків краще встановити підвищену температуру, наприклад, temperature у діапазоні 0.7 – 1.0. У цій задачі потрібна креативність, різноманітність і оригінальність ідей, щоб пропонувати багато унікальних та не шаблонних варіантів.

3. Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?

За характером впливу `temperature` масштабує загальний розподіл ймовірностей для всього словника, роблячи менш імовірні слова більш доступними, тоді як `top_r` працює через динамічне відсікання хвоста розподілу: він залишає лише той набір слів, сукупна ймовірність яких у сумі дорівнює заданому порогу, повністю відкидаючи весь решток абсолютно маловірогідних або випадкових токенів, незалежно від їхньої кількості.

4. Чи означає `temperature=0` повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

Ні, `temperature=0` не завжди гарантує абсолютну 100% відтворюваність результату на рівні бітів у всіх середовищах. Хоча логічно модель має обирати найбільш імовірний токен, особливості паралельних обчислень на плаваючій комірці у різних апаратних конфігураціях або потоках іноді можуть призводити до мікроскопічних зсувів у розрахунках ймовірностей. Проте в експерименті з 10 запусків зазвичай спостерігається ідеальна стабільність: усі відповіді збігаються.

5. Наведіть найбезглуздішу відповідь, яку модель видала при `temperature=1.5`, і поясніть, чому вона взагалі могла з'явитися.

Найбезглуздішою відповіддю стала розгорнута галюцинація про «братів Уолтера Маршала... стрімі кариб на кіно іграв... кінопотентіянALI, він з'являється першим після їхніх родичів: в листопаду на фонде Меркулієн з'являє своє перстентя...» та згадки «Ночної Січі». Вона з'явилася тому, що за дуже високої температури шкала ймовірностей слів «розмивається», і модель починає обирати випадкові, малопов'язані між собою токени з далеких гілок словника.

Крок 8:

1. Що саме математично робить параметр `temperature` із розподілом ймовірностей наступного токена?

Математично параметр `temperature` (T) ділить логіти (нерозподілені оцінки виходу моделі z_i) перед застосуванням функції Softmax. Формула виглядає так:

$$P(i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}.$$

Коли температура прямує до нуля ($T \rightarrow 0$), різниця між логітами збільшується, через що ймовірність найімовірнішого токена прагне до одиниці, а решти — до нуля. При високих значеннях (наприклад, $T > 1$) логіти діляться на велике число, що згладжує розподіл, роблячи ймовірності всіх токенів більш рівномірними.

2. Чим `top_p` відрізняється від `temperature` — обидва впливають на різноманітність, але за яким механізмом?

1. Вони працюють за принципово різними механізмами:
 - **Temperature** змінює «крутизну» всього глобального розподілу ймовірностей, роблячи менш імовірні слова більш або менш доступними для вибору на кожному кроці.
 - **Top_p (Nucleus sampling)** використовує динамічне відсікання хвоста розподілу. Токени сортуються за ймовірністю у спадаючому порядку, і модель бере лише ту мінімальну підмножину слів, сукупна сума ймовірностей якої досягає заданого порогу p (наприклад, 0.9 або 90%). Усі інші маловірогідні слова з «довгого хвоста» повністю виключаються незалежно від їх кількості.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

На практиці інженери зазвичай рекомендують змінювати **тільки один із цих параметрів** для однієї генерації (найчастіше `temperature`), залишаючи інший за замовчуванням (або `top_p = 1.0`). Зміна обох параметрів одночасно ускладнює контроль над поведінкою моделі, оскільки вони накладаються один на одного: `top_p` може відсікти ті варіанти, які `temperature` намагалася зробити доступнішими, що часто призводить до непередбачуваних результатів.

4. Чому навіть при `temperature=0` мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Навіть при `temperature = 0` (коли вибір завжди зводиться до найімовірнішого токена) повна детермінованість не гарантується через особливості паралельних обчислень на плаваючій комійці (floating-point arithmetic) у сучасних GPU та CPU. Через неасоціативність паралельних операцій у великих матричних множеннях на різних апаратних конфігураціях або при зміні черги потоків усередині бекенду виникають мікроскопічні похибки округлення. Вони можуть змінити сортування рівнозначних логітів, що призведе до вибору іншого токена (хоча у вашому експерименті з 10 запусків зазвичай спостерігається ідеальна стабільність: усі відповіді збігаються).

5. Які значення `temperature` використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту

Для витягу структурованих даних (JSON, класифікація, виділення сутностей):

Використовують нульову або близьку до нуля температуру ($T = 0.0$), оскільки тут критично важлива точність, дотримання формату та відсутність «творчості» чи вигадок.

Для генерації творчого тексту (оповідання, заголовки, брейнштормінг, діалоги):

Використовують підвищені значення (T в діапазоні від 0.7 до 1.0 або вище), щоб забезпечити високу різноманітність лексики, оригінальність і нешаблонні поєднання слів.

Висновок по роботі:

Критерій	Локальна мала	Локальна більша	Хмарна
Швидкість генерації	Середня	Нижня	Висока
TTFT	Середня	Більша на старті	Мінімальна
Якість, балів 12	3	1	7
Стабільність формату	Низька	Низька	Середня
Потрібні ресурси	4-8 ГБ RAM	8-16 ГБ RAM	Уся обробка запитів на стороні хмарного провайдера
Вартість	Безкоштовно	Безкоштовно	Ліміти
Куди йдуть дані	Нікуди	Нікуди	Хмарний провайдер
Працює без інтернету	Так	Так	Ні
Коли обирати саме її	Краще обрати для завдань, де важлива конфіденційність даних та функціонал за відсутності інтернету, а також для коротких нескладних запитань.	Ті ж самі причини, що і для меншої локальної моделі, проте є різниця у складності запитів. До цієї моделі можна звертатися для складних питань, де потрібно більше знань чи логіки.	Вибирати тільки за умови впевненості, що буде стабільний інтернет.